

CS 230 Distributed Computer Systems

Homework 2

1 The Approach

In this assignment, we need two programs to simulate the communication process between two devices. Let's name the two programs after sender and receiver. To transport message between devices, we need socket APIs provided by Python in both sender and receiver programs.

```
import socket, sys
```

1.1 Sender Side Construction

When implementing the `main` function, we should do two things: build the socket, which serves as a bridge passing message between the sender and the receiver; After built up the socket, we can begin to listen to the receiver to get receiver's information and send data to the receiver.

1.1.1 Building the Socket

At first, we can initialize a listening socket for sender utilizing Python's socket APIs. Here, the `AF_INET` refers to the IPv4 address, the `SOCK_STREAM` means connection oriented TCP protocol.

```
sender_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Then, we should bind the socket to some IP address and well-known ports. If we want to bind to all available IP addresses, we should specify `0.0.0.0` as our IP address, which will make the routing in this program easier. I also choose the port number 8080.

```
sender_socket.bind(('0.0.0.0', 8080))
```

Since we are building a one-to-one sender-receiver model, we will set the maximum connection number of the `sender_socket` to 1.

```
sender_socket.listen(1)
```

Now we can say the socket of the sender is built up and open to use.

1.1.2 Communication with the Receiver

Here, I build a thread to execute the process. The further implementation will be inside the following loop:

```
while True:
```

By listening to the receiver, we can get the receiver's connection and address.

```
receiver_connection, receiver_address = sender_socket.accept()
```

Then, since we only need to transport a short string, the size of the string won't be too large. We can set the maximum size of the receiver's data to 1024. By the mean time, we can get the receiver's data here.

```
data = receiver_connection.recv(1024)
```

Now we get the receiver's data. In order to check if the process is successful, we can print the data to the console.

```
print("Data from the Receiver: %s" % (data))
```

After listening to the receiver, we can send our message to the receiver through the socket.

```
message = "Good"  
receiver_connection.send(message.encode())
```

After sending the message, we can close the socket now.

```
receiver_connection.close()
```

At this stage, the thread is ended. We can close the sender and quit the program now.

```
sender_socket.close()  
sys.exit()
```

1.2 Receiver Side Construction

To build the receiver program, the general steps are analogous to the sender side construction. Here is the way to implement the `main` function. At first we build the socket between sender and receiver, then send request to the sender, after sender sent its message, we retrieve that message then close our receiver.

1.2.1 Building the Socket

At first, we should initialize the socket:

```
receiver_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Then, by convenience, we can set the public IPv4 address to 10.211.55.4 (ip address of interface `enp0s5` on my Ubuntu Linux), and port number 8080, and connect the socket.

```
receiver_socket.connect(('10.211.55.4', 8080))
```

1.2.2 Communication with the Sender

After building the socket, we can send our request to the sender to get its message.

```
request = "GET MSG"  
receiver_socket.send(request.encode())
```

By then, if the sender get our request, it will send a message to the receiver, now we are going to retrieve the message from the sender:

```
message = receiver_socket.recv(1024).decode()
```

To see if we get the message successfully, we can print the message out on the console:

```
print("Message from the Sender %s" % (message))
```

Now we accomplished the communication between sender and server, we can close the receiver socket and quit the program:

```
receiver_socket.close()  
sys.exit()
```

2 Wireshark Packet Analysis

2.1 Building the Environment

I deployed the `sender.py` program on a Ubuntu Linux 18.04 device(Ubuntu), and the `receiver.py` program on a macOS Mojave 10.14.2 device(Mac).

From the Ubuntu, we can start wireshark by running shell script:

```
wireshark
```

We can get the IP address of the Ubuntu by:

```
ifconfig
```

So now we get the IP address of the Ubuntu, which is 10.211.55.2. With this address, we can add a capture filter entry in wireshark to isolate the packets corresponding to the communication between the `sender.py` and the `receiver.py`.

```
To 10.211.55.2 | ip.dst == 10.211.55.2
```

With wireshark set up, we can start `sender.py` on Ubuntu:

```
python2 sender.py
```

Then, start `receiver.py` on Mac:

```
python2 receiver.py
```

2.2 Network Analysis

After setting up the environment and launching the sender and receiver, we can get the captured packets in wireshark.

10.211.55.4	TCP	78	59069 → 8080	[SYN, ECN, CWR] Seq=0 Win=65535 Len=0 MSS=1460 W...
10.211.55.2	TCP	74	8080 → 59069	[SYN, ACK, ECN] Seq=0 Ack=1 Win=28960 Len=0 MSS=...
10.211.55.4	TCP	66	59069 → 8080	[ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=1100885...
10.211.55.4	TCP	73	59069 → 8080	[PSH, ACK] Seq=1 Ack=1 Win=131712 Len=7 TSval=11...
10.211.55.2	TCP	66	8080 → 59069	[ACK] Seq=1 Ack=8 Win=29056 Len=0 TSval=49226088...
10.211.55.2	TCP	70	8080 → 59069	[PSH, ACK] Seq=1 Ack=8 Win=29056 Len=4 TSval=492...
10.211.55.4	TCP	66	59069 → 8080	[ACK] Seq=8 Ack=5 Win=131712 Len=0 TSval=1100885...
10.211.55.2	TCP	66	8080 → 59069	[FIN, ACK] Seq=5 Ack=8 Win=29056 Len=0 TSval=492...
10.211.55.4	TCP	66	59069 → 8080	[ACK] Seq=8 Ack=6 Win=131712 Len=0 TSval=1100885...
10.211.55.4	TCP	66	59069 → 8080	[FIN, ACK] Seq=8 Ack=6 Win=131712 Len=0 TSval=11...
10.211.55.2	TCP	66	8080 → 59069	[ACK] Seq=6 Ack=9 Win=29056 Len=0 TSval=49226088...

By opening the fifth packet, we can see the byte information inside the packet. Using the interpreter inside Wireshark, we can answer the following questions.

1. What is encoded in bytes 0-5 and 6-11?

0-5: Destination MAC address (Globally unique address and Individual address)

```
Address: Parallel_00:00:08 (00:1c:42:00:00:08)
```

6-11: Source MAC address

```
Address: Parallel_27:07:24 (00:1c:42:27:07:24)
```

2. What is encoded in, and what is the relationship between, byte 14 and the two bytes 16-17?

14: The header length of the message

```
0101 = Header Length: 20 bytes (5)
```

16-17: Total length of the message

```
Total length: 52
```

Header is pointing to the beginning of the data, header length is the length of the header. While total length is the length of the whole datagram. Header length is a part of the total length.

3. What is encoded in bytes 18-19?

18-19: Identification of IPv4

```
Identification: 0x1b12(6930)
```

4. What is encoded in bytes 20-21?

20-21: Fragment offset of the Flag of the IPv4

```
...0 0000 0000 0000 = Fragment offset: 0
```

5. What is encoded in byte 23?

23: Protocol Type

```
Protocol: TCP(6)
```

6. What is encoded in bytes 26-29 and 30-33?

26-29: Source address in the header

Source: 10.211.55.4

30-33: Destination address in the header

Destination: 10.211.55.2

7. What is encoded in bytes 34-35 and 36-37?

34-35: Source port

Source Port: 8080

36-37: Destination Port

Destination Port: 59069

8. What is encoded after byte 65?

66-...: App header and user data

NULL

2.3 Layer Classification

2 Link Layer

0-5

6-11

3 Network Layer

14

16-17

18-19

20-21

23

26-29

30-33

4 Transport Layer

34–35

36–37

7 Application Layer

66–...

3 File Listing

In Ubuntu, the structure of key files of this project can be interpreted as follows:

sender.py

\packets

capture.pcapng

In Mac, the structure of key files of this project can be interpreted as follows:

receiver.py